Assignment 3: Designing methods for Complex Data, Practice with Lists and Trees, Drawing

Goals: Practice working with lists. Learn to design methods and practice designing with accumulators. Learn to use the image library.

3.1 Instructions

Be very, very careful with naming! Again, the solution files expect your submissions to be named a certain way, so that they can define their own `Examples` class with which to test your code. Therefore, whenever the assignment specifies:

- the names of classes,
- the names and types of the fields within classes,
- the names, types and order of the arguments to the constructor,
- the names, types and order of arguments to methods, or
- filenames,

...be sure that your submission uses exactly those names.

Make sure you follow the style guidelines that handin enforces. For now the most important ones are: using spaces instead of tabs, indenting by 2 characters, following the naming conventions (data type names start with a capital letter, names of fields and methods start with a lower case letter), and having spaces before curly braces.

You will submit this assignment by the deadline using the handin submission system. You may submit as many times as you wish. Be aware of the fact that close to the deadline the system may have a long queue of submissions which means it takes longer for your code to be submitted - so try to finish early.

There will be a separate submission for each problem - it makes it easier to grade each problem, and to provide you with the feedback for each problem you work on.

Practice Problems

Work out these problems on your own. Save them in an electronic portfolio, so you can show them to your instructor, review them before the exam, use them as a reference when working on the homework assignments.

- Problems 18.1 - 18.4 on page 225
- Problem 18.5 on page 229
- Problem 18.6 on page 234
- Problem 19.4 on page 263

Problem 1: Understanding the `String` class

Related files:
Strings.java

For all questions in this problem, be sure to follow the design recipe carefully:

- Give sufficient examples of data, and sufficient tests, to test your methods thoroughly.
- If you find yourself wanting to use a field-of-field, stop. Fill out the template for each method, and figure out another design.
- Think carefully about how to use dynamic dispatch, and where to define methods, to keep your code as simple and clean as possible.

Note: The following method defined for the class `String` may be useful:

```
// does this String come before the given String lexicographically?
// produce value < 0   --- if this String comes before that String
// produce value zero  --- if this String is the same as that String
// produce value > 0   --- if this String comes after that String
int compareTo(String that)
```

- > When referring to strings, this is known as a "case-insensitive" sort, since when it examines two strings, it converts everything to lowercase (or uppercase — would it make any difference?) *before* comparing them, so the comparison can't use—or be sensitive to—differences in casing.

  Design the method `sort` that produces a new list, sorted in alphabetical order, treating all `String`s as if they were given in all lowercase.

  Note: The `String` class defines the method `toLowerCase` that produces a `String` just like the one that invoked the method, but with all uppercase letters converted to lowercase.

- Design the method `isSorted` that determines whether this list is sorted in alphabetical order, in a case-insensitive way.

  Hint: You will likely need a helper method. You may want to review the accumulator style functions we have seen in DrRacket.

- Design the method `interleave` that takes this list of `String`s and a given list of `String`s, and produces a list where the first, third, fifth... elements are from this list, and the second, fourth, sixth... elements are from the given list. Any "leftover" elements (when one list is longer than the other) should just be left at the end.

- Design the method `merge` that takes this sorted list of `String`s and a given sorted list of `String`s, and produces a sorted list of `String`s that contains all items in both original lists, including duplicates. (This is *not* the same computation as for `interleave`, but the two methods are similar. Can you construct an example of two lists such that interleaving them and merging them produce different results? Can you construct another example where the two results are the same?)

- Design the method `reverse` that produces a new list of `String`s containing the same elements as this list of `String`s, but in reverse order.

  Hint: The cleanest solution to this problem uses a helper method, in a style seen already in this problem.

- Design the method `isDoubledList` that determines if this list contains pairs of identical strings, that is, the first and second strings are the same, the third and fourth are the same, the fifth and sixth are the same, etc.

  Hint: Think carefully about how to test this method.

- > This isn't the same as the typical definition of palindromes, which asks whether a single string contains the same letters when read in either order.

  Design the method `isPalindromeList` that determines whether this list contains the same words reading the list in either order.

  Hint: Several of the methods defined above will be helpful in your solution.

Submit your work in the completed `Strings.java` file.

WARNING: Be extra careful not to name your file "String.java" – or else Java will think that you're trying to redefine the built-in String class, and everything will break in creatively bizarre ways.

Problem 2: Drawing trees

In this problem, you will be drawing and working with trees.

```
import java.awt.Color;

interface ITree { /* see methods below */ }

class Leaf implements ITree {
  int size; // represents the radius of the leaf
  Color color; // the color to draw it
}

class Stem implements ITree {
  // How long this stick is
  int length;
  // The angle (in degrees) of this stem, relative to the +x axis
  double theta;
  // The rest of the tree
  ITree tree;
}

class Branch implements ITree {
  // How long the left and right branches are
  int leftLength;
  int rightLength;
  // The angle (in degrees) of the two branches, relative to the +x axis,
  double leftTheta;
  double rightTheta;
```

```
  // The remaining parts of the tree
  ITree left;
  ITree right;
}
```

A `Stem` at an angle of 90 degrees is growing straight up; a `Branch` with a left angle of 135 degrees and a right angle of 45 degrees points on both upward diagonals. (Leaves don't need angles, since we can just approximate them with circles.)

Hints: To design many of the methods for this problem, it helps to refresh some basic trigonometry.

- The reference for angles is usually the +X axis.
- Let us say we given a point (x,y). Join this point to the origin, creating a line of length l and an angle θ in radians. Then $x=l\cos\theta$, and $y=l\sin\theta$. In Java you can compute cosines and sines using `Math.cos` and `Math.sin` respectively.

For each method, it will help to visualize or draw on paper what you are trying to achieve before you attempt to design and write code.

- Design the method `WorldImage draw()`, that renders your `ITree` to a picture. Draw all branches and stems using `LineImage`s (this will make the coordinate manipulations easier than if you use rectangles) of whatever color you choose. Understanding the concept of pinholes in your image will be very helpful, so make sure to read the quick-start guide below, and the documentation for more information. **Try to draw some simple images unrelated to this problem to get a feel for the library, before writing this specific method.**

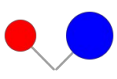
  You do not have to write extensive tests for this method, but you should write at least some simple sanity checks. See below for more information about getting started with the image library. We will not write automated tests for this method, but will grade it manually.

- Design the method `boolean isDrooping()`, that computes whether any of the twigs in the tree (either stems or branches) are pointing downward rather than upward.

- Design the method
  `ITree combine(int leftLength, int rightLength double leftTheta, double rightTheta, ITree otherTree)`
  This method takes the current tree and a given tree and produces a `Branch` using the given arguments, with this tree on the left and the given tree on the right... but with a twist, literally.

  Here are two trees, with the stems and branches drawn in different colors so we can keep them apart:

  ```
  tree1 = new Branch(30, 30, 135, 40, new Leaf(10, Color.RED), new Leaf(15, Color.BLUE))
  ```

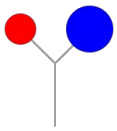
  

  ```
  tree2 = new Branch(30, 30, 115, 65, new Leaf(15, Color.GREEN), new Leaf(8, Color.ORANGE))
  ```

  If we attach each of them to stems growing at 90 degrees (i.e., straight up), we get:

  ```
  new Stem(40, 90, tree1)
  ```

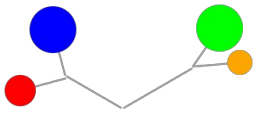


```
new Stem(50, 90, tree2)
```



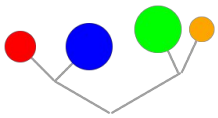
Now suppose we want to attach those two stems to form a branch, with the left branch at 150 degrees and the right branch at 30 degrees (for example). We would get

```
tree1.combine(40, 50, 150, 30, tree2)
```



Your method should implement this combination step. Note that this result is different from simply placing both trees in a branch directly:

```
new Branch(40, 50, 150, 30, tree1, tree2)
```



You can deduce why this is from the interpretations of the various fields in the data definitions above, and design helper methods to implement the desired behavior accordingly.

- Design the method `double getWidth()` that returns the width of the tree. Assume that leaves are drawn as circles, and their size is used as their radius.

What to submit

Please submit your solution in a file named `Tree.java`, and name your examples class `ExamplesTree`.

Using the javalib library

The *javalib* library provides the support for the design of interactive games and creating images composed by combining geometric shapes as well as image files. See The Image Library for more information.

To use the library, download the `javalib` file above and add it to your project the same way you have added the tester library.

At the top of the `.java` file where the library is used, add the following import statements:

```
import tester.*;                // The tester library
import javalib.worldimages.*;   // images, like RectangleImage or OverlayImages
import javalib.funworld.*;      // the abstract World class and the big-bang library
import java.awt.Color;          // general colors (as triples of red,green,blue values)
                                // and predefined colors (Color.RED, Color.GRAY, etc.)
```

You can test images using `check-expect` just as you can check other values:

```
boolean testImages(Tester t) {
  return t.checkExpect(new RectangleImage(30, 20, OutlineMode.SOLID, Color.GRAY),
                       new RectangleImage(30, 20, OutlineMode.SOLID, Color.GRAY));
}
```

But note that you must construct the images in the same manner: for example, the following test will fail:

```
boolean testFailure(Tester t) {
  return t.checkExpect(
      new ScaleImageXY(new RectangleImage(60, 40, OutlineMode.SOLID, Color.GRAY), 0.5, 0.25),
      new RectangleImage(30, 15, OutlineMode.SOLID, Color.GRAY));
}
```

Finally, you can display your images so that you can see whether you're on the right track, as follows:

```
boolean testDrawTree(Tester t) {
  WorldCanvas c = new WorldCanvas(500, 500);
  WorldScene s = new WorldScene(500, 500);
  return c.drawScene(s.placeImageXY(myTree.draw(), 250, 250))
      && c.show();
}
```

See The Image Library for more information.